

Cloud and DevOps

Contents

What DevOps is

1. [DevOps and what it replaced](#)
2. [DORA metrics](#)

Cloud fundamentals

3. [Service models and shared responsibility](#)
4. [Regions, zones and placement](#)
5. [Cloud networking](#)
6. [The cost model](#)

Containers and orchestration

7. [What a container is](#)
8. [Building images](#)
9. [Kubernetes essentials](#)
10. [When Kubernetes is the wrong answer](#)

Delivery

11. [Continuous integration, delivery and deployment](#)
12. [Designing a pipeline](#)
13. [Release strategies](#)
14. [Infrastructure as code](#)
15. [Configuration and secrets](#)

Operating

16. [Service levels and signals](#)
17. [Alerting and on-call](#)
18. [Incident response](#)
19. [Error budgets and toil](#)

Reference

20. [Interview questions](#)
21. [Common mistakes](#)

1. DevOps and what it replaced

DevOps is a set of practices aimed at shortening the time between a change being written and it running safely in production, without trading away quality to get there.

It is easier to define by what it replaced. Development and operations used to be separate organisations with opposing incentives: development was rewarded for shipping change, operations for keeping things stable, and change is the enemy of stability. Every release became a negotiation, releases grew large because they were rare, and large releases were riskier, which made them rarer still.

The practices that break that loop:

- **Shared ownership.** The team that builds a service runs it, so the cost of operability lands on the people who can fix it.
- **Automation of the path to production,** so releasing is routine rather than an event.
- **Small, frequent changes,** because a small change is easier to review, verify and reverse.
- **Production feedback reaching developers,** through metrics, logs and being on call.

Two precisions worth having ready.

DevOps is primarily a set of practices and organisational principles, rather than the name of a separate team. A "DevOps team" sitting between development and operations has rebuilt the handoff the practice exists to remove, and the useful version of that function is a **platform team** providing self-service tooling, which the software architecture note covers under Team Topologies. The job title obviously exists everywhere, and arguing about it in an interview is rarely worth the time.

SRE, Site Reliability Engineering, is Google's reliability-focused way of applying many of the same principles, with specific mechanisms attached: service level objectives, error budgets, and a cap on toil. Section 19 covers those.

2. DORA metrics

DORA, DevOps Research and Assessment, identified four measures of software delivery performance, and they are worth knowing because they get asked as a question about how to measure a team.

Metric	What it measures
Deployment frequency	How often changes reach production
Lead time for changes	How long from commit to running in production
Change failure rate	What proportion of deployments cause a degradation needing remediation
Time to restore service	How long to recover once something breaks

The first two measure speed, the last two measure stability. The finding that makes them interesting is that **those are not opposed**: teams that score well on speed tend to score well on stability too. That contradicts the intuition that shipping faster necessarily breaks more, and the causal story is that small frequent changes are easier to verify and easier to reverse.

The original model had those four delivery metrics. **Reliability** was added later as a fifth, covering operational performance against expectations. The terminology has also shifted, so time to restore service appears in newer material as failed deployment recovery time; the measurement is the same.

Two qualifications worth stating rather than waiting to be challenged on. These measure **delivery performance**, not whether the software is worth building or whether the code is good. And they are **gameable**: deployment frequency rises immediately if one deploy is split into ten, and lead time falls if work is not counted until it starts. They are useful as a trend for a team looking at its own numbers, and poor as a target imposed on it.

3. Service models and shared responsibility

The service models describe where the line falls between what the provider manages and what we do.

	Provider manages	We manage
On premises	Nothing	Everything, from the building to the application
IaaS, Infrastructure as a Service	Hardware, virtualisation, network	Operating system, runtime, application, data
PaaS, Platform as a Service	The above plus operating system and runtime	Application and data
FaaS, Functions as a Service	The above plus scaling and instance lifecycle	Function code and data
SaaS, Software as a Service	Everything	Configuration and data

Moving down the list trades control for operational load. An IaaS virtual machine can be tuned in any way, and needs patching, monitoring and capacity planning. A PaaS deployment gives up that control and removes most of that work.

FaaS is more than PaaS with the lifecycle removed. It also brings an event or request-driven execution model, an expectation that the function holds no state between invocations, execution time limits, cold starts when an idle function is invoked, and billing by invocation and duration rather than by the hour.

The shared responsibility model is the security framing of the same line, and misunderstanding it causes real breaches. The provider is responsible for the security **of** the cloud: physical facilities, the hypervisor, the managed service internals. The customer is responsible for security **in** the cloud: identity and access

configuration, network rules, encryption choices, patching anything they manage, and what their application does.

A storage bucket left open to the internet is the customer's responsibility. So is an over-permissive access role, which is the one that turns a small compromise into a large one.

4. Regions, zones and placement

A **region** is a geographic area. An **availability zone** is one or more isolated data centres within a region, with independent power, cooling and networking, connected to the other zones in that region by low-latency links.

That structure gives three placement choices:

Scope	Protects against	Costs
Single zone	Nothing beyond individual machine failure	Cheapest, and a zone failure is an outage
Multi-zone	One data centre failing	Cross-zone data transfer charges, slightly higher latency
Multi-region	A whole region failing, and distance to distant users	Replication lag, data residency questions, substantially more operational work

Multi-zone is a common default for production workloads where a zone failure must not cause an outage, because the failure it protects against is real and the cost is modest. Whether it is right for a given system still depends on recovery objectives, criticality and what the data layer supports.

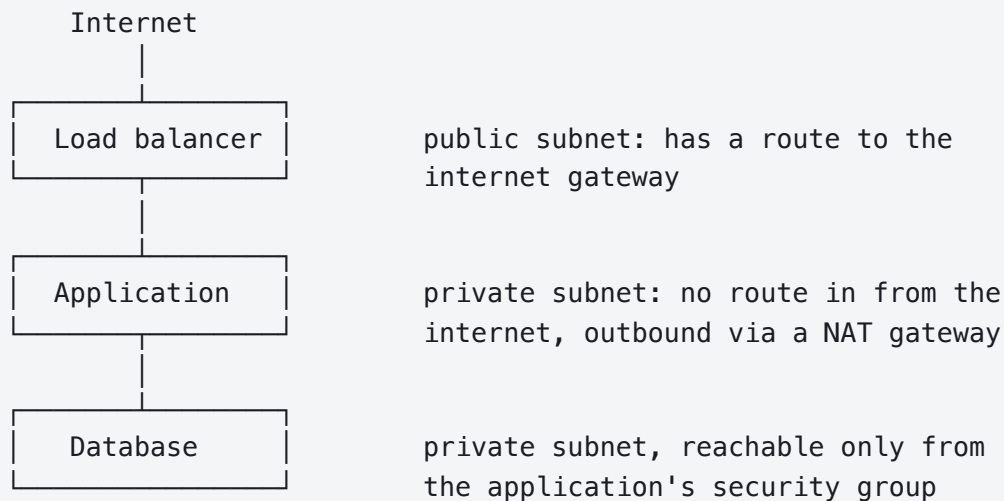
Multi-region is a much larger step, and the system design note covers the consistency questions it brings. One common design is single-region writes with reads served globally; active-active is possible and brings harder conflict resolution with it.

Data residency is the constraint that overrides all of this. When regulation requires that data about people in a jurisdiction stays in that jurisdiction, placement stops being a performance decision and becomes a legal one, which usually forces regional partitioning of the data model itself.

Edge locations are a separate tier, holding cached content close to users, which the caching note covers.

5. Cloud networking

The shape almost every cloud deployment takes, and the vocabulary needed to describe it:



Component	What it does
VPC or VNet	A private network in the provider's cloud, with an address range we choose
Subnet	A slice of that range, tied to one availability zone
Route table	Decides where traffic from a subnet goes, and is what makes a subnet public or private
Internet gateway	Allows traffic to and from the internet for a public subnet
NAT gateway	Lets private resources make outbound connections without being reachable inbound
Security group	A stateful firewall attached to a resource, usually allow-rules only
Network ACL	A stateless firewall at the subnet level, allowing and denying
Load balancer	Distributes traffic across instances, usually the only public entry point
Private endpoint	Reaches a managed service without traffic leaving the private network

Three points worth carrying into a design discussion.

Public and private is a routing property, not a setting. A subnet is public because its route table sends internet-bound traffic to an internet gateway. Nothing else distinguishes them.

Security groups are stateful. A reply to an allowed inbound connection is allowed out automatically, so there is no need for a matching outbound rule. Network ACLs are stateless and do need both directions, which is a classic source of confusing failures.

The default should be private. Only the load balancer needs to be publicly reachable, and databases in particular belong in a private subnet with access restricted to the application's security group rather than to an address range.

6. The cost model

Cost is a quality attribute, in the sense the software architecture note uses, and it is the one engineers most often leave out of a design.

What drives a cloud bill:

- **Compute time**, charged per instance-hour or per request-duration.
- **Storage volume**, usually per gigabyte-month, with tiers for data accessed rarely.
- **Data transfer**, which is where the surprises live.
- **Managed service premiums**, the difference between running a database and having one run.
- **Idle resources**, which cost the same as busy ones.

Data transfer is the line item that catches people. Inbound traffic from the internet is often free, while outbound traffic and traffic crossing zones or regions commonly carries a charge, and the specifics vary by provider and by service path. A design that spreads chatty services across zones for availability can generate a meaningful bill purely in cross-zone traffic, which is an argument for zone-aware routing rather than against multi-zone deployment.

Pricing models are worth knowing by shape:

Model	Trade
On-demand	Highest unit price, no commitment, use it as needed
Reserved or committed use	Substantial discount for committing to a term or a spend
Spot or preemptible	Much cheaper, and can be reclaimed with little notice

Spot capacity suits work that can be interrupted and retried, such as batch processing or a stateless worker pool, and does not suit anything holding state that cannot be rebuilt.

The practices that keep cost visible: tagging resources so spend can be attributed to a team or service, rightsizing rather than guessing instance sizes, autoscaling **down** as well as up, and deleting orphaned resources such as unattached volumes and idle load balancers.

7. What a container is

A container is **not a lightweight virtual machine**, and the difference explains most of its behaviour. A container is an ordinary process running on the host's kernel, isolated by two Linux features:

- **Namespaces** control what the process can *see*: its own process identifiers, network interfaces, mount points, users and hostname.
- **Control groups**, or cgroups, control what the process can *use*: processor time, memory, and input and output bandwidth.

An image adds the third piece, a layered filesystem, so the process sees its own root filesystem assembled from stacked read-only layers with a writable layer on top.

	Virtual machine	Container
Isolation boundary	Virtualised hardware	Kernel features
Kernel	Its own	Shared with the host
Start time	Boots an operating system	Generally much faster, though it still depends on image size and application startup
Overhead	A full operating system per instance	The process, plus a little
Isolation strength	Stronger	Weaker, since a kernel vulnerability crosses the boundary

The JVM in a container

A Java-specific trap worth knowing. Older JVMs read the *host's* memory and processor count rather than the cgroup limit, so a container capped at 512 MB could see the machine's 64 GB and size its heap for that, then get killed by the kernel. Container awareness has been on by default since JDK 10 and was backported to 8u191, so modern images behave, but the lesson behind it still applies.

Set the heap as a proportion rather than an absolute: `-XX:MaxRAMPercentage=75`. Do not set `-Xmx` equal to the container limit, because the JVM also needs metaspace, thread stacks, code cache and direct buffers outside the heap, and exceeding the container limit does not raise `OutOfMemoryError` — the kernel kills the process, which surfaces as a pod terminating with exit code 137 and no Java stack trace to explain it.

8. Building images

Three practices carry most of the value.

Order instructions from least to most frequently changing, because each instruction is a cached layer and everything after a changed layer rebuilds. Copying the whole source tree before resolving dependencies means every source edit re-downloads every dependency.

Use a multi-stage build, so the toolchain used to compile does not ship to production. The build stage has the JDK and the build tool; the final stage has a runtime and the artifact.

Run as a non-root user, because running the process as root increases the damage a container escape or an isolation failure can do.

```
# Build stage: full toolchain, none of which ships.
FROM eclipse-temurin:21-jdk AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -B dependency:go-offline          # cached unless pom.xml changes
COPY src ./src
RUN mvn -B package -DskipTests

# Runtime stage: only what is needed to run.
FROM eclipse-temurin:21-jre
WORKDIR /app
RUN useradd --create-home --shell /bin/false app
USER app
COPY --from=build /app/target/app.jar .
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "app.jar"]
```

Four more rules that matter:

- **Never bake secrets into an image.** A secret in any layer stays in the image history even if a later layer deletes the file, and anyone who can pull the image can read it.
- **Tag with something immutable**, such as a commit identifier or a digest. Deploying `latest` means the thing tested and the thing running may differ with no record of it.
- **Prefer small base images**, because fewer packages means a smaller attack surface and faster pulls. Alpine is popular for size, and it uses the musl C library rather than glibc, which can cause compatibility or performance differences with native dependencies and JVM tooling, so a slim Debian-based or distroless image is often the safer default for Java.
- **Scan images** for known vulnerabilities in the pipeline, and rebuild rather than patch in place.

9. Kubernetes essentials

Kubernetes runs containers across a set of machines, keeping actual state matching declared state. The objects worth knowing:

Object	What it is
Pod	One or more containers sharing a network namespace and storage. The unit of scheduling, and usually one container
Deployment	Keeps N identical pods running and performs rolling updates by managing ReplicaSets
Service	A stable address and DNS name for a changing set of pods
Ingress	Declares HTTP routing from outside the cluster to Services. An Ingress Controller is what implements it
ConfigMap	Non-secret configuration, injected as environment variables or files
Secret	The same, intended for credentials, though see section 15

Object	What it is
Namespace	A logical partition for grouping and access control

Requests and limits are the setting most often got wrong, and the consequences differ by resource:

```
resources:
  requests:           # what the scheduler uses to decide placement
    cpu: "250m"
    memory: "512Mi"
  limits:             # the ceiling once it is running
    cpu: "1000m"
    memory: "1Gi"
```

Requests are the amounts the scheduler uses when choosing a node and calculating what capacity remains. Limits are the ceiling once the container runs, and the two resources behave differently at that ceiling.

Exceeding the **processor** limit throttles the container, so it runs slowly. Exceeding the **memory** limit can cause the kernel to kill the process, because memory cannot be throttled the way processor time can. That asymmetry is why memory limits deserve more care, and why the JVM percentage setting from section 7 matters.

Requests set too low let the scheduler place more work on a node than it can carry, so pods are evicted when it comes under pressure. Requests set too high reserve capacity nothing else can use.

Probes decide what the platform does when a container is unhealthy, and the distinction matches the one in the API note:

Probe	Failing means
Liveness	Restart the container
Readiness	Stop sending it traffic, leave it running
Startup	Still starting, hold off the liveness probe

The mistake to avoid: **a liveness probe that checks a downstream dependency**. When that dependency has a brief outage, every replica fails liveness at once and the platform restarts all of them, turning a dependency blip into a full outage. Dependency health belongs in readiness, or nowhere.

Horizontal Pod Autoscaler adds and removes pods based on a metric. Processor use is the default and often the wrong signal for a service that spends its time waiting, where in-flight concurrency, queue depth or request latency tracks real load better.

10. When Kubernetes is the wrong answer

Worth having ready, because it demonstrates judgment rather than familiarity.

Kubernetes carries real operational cost: cluster lifecycle and upgrade management, networking and ingress configuration, access control, resource tuning, and a large surface area that every engineer touching it has to learn.

It earns that cost when there are **many services and several teams needing to deploy independently**, when bin-packing many workloads onto shared nodes saves real money, or when there is a platform team whose job is to run it.

It usually does not earn it for a handful of services owned by one team with steady load. Managed container platforms such as ECS, Cloud Run or App Runner, or a platform service, deliver most of the deployment and scaling benefit for a fraction of the operational surface. The honest position is that Kubernetes is a good answer to a problem many organisations adopting it do not yet have.

11. Continuous integration, delivery and deployment

Three terms, and the last two get used interchangeably when they are different.

Continuous integration means frequent integration into a shared mainline, with every merge verified automatically by a build and a test suite. Practitioners usually aim for at least daily, though the definition is about frequency and automated verification rather than a specific cadence. The value is not the automation, it is the frequency: integrating small changes continuously removes the large, painful merge at the end of a long-lived branch.

Continuous delivery means every change that passes the pipeline is **releasable**. The artefact is built, tested and ready, and deploying is a business decision taken by pressing a button.

Continuous deployment means every change that passes the pipeline **is deployed**, with no human gate.

The gap between the last two is a decision about whether a human should approve each release, and continuous delivery is the more common resting point in regulated or high-consequence environments.

Trunk-based development is the branching model that makes continuous integration real: short-lived branches merged within a day or two, rather than feature branches living for weeks. The obvious objection is what to do with work that is not finished, and the answer is **feature flags**, which let incomplete code merge while staying switched off. The cost is that flags need a lifecycle, because a flag nobody removes becomes a permanent branch in the code.

12. Designing a pipeline

The principle that matters most: **build once, then promote the same artefact**. Rebuilding for each environment means the thing tested is not the thing shipped, and the difference between them is exactly where a problem will hide.

```
commit
  ↓ compile, unit tests, static analysis      fast, cheap, run first
  ↓ package the artefact                    built once, here
  ↓ dependency and image scanning
  ↓ integration tests against real dependencies
  ↓ deploy to staging, smoke test
  ↓ deploy to production
  ↓ verify: health checks, smoke tests, watch the metrics
  ↓ roll back or forward if the signals are bad
```

The last two steps are part of the pipeline rather than something that happens afterwards. A deployment that completes and is never verified has only proved that the artefact installed.

Order the stages by cost and speed so failures surface early. A pipeline that runs a twenty-minute integration suite before a compile error is found wastes the most expensive resource involved, which is the engineer's attention.

Three properties of a pipeline worth insisting on:

- **Reproducible.** The same commit produces the same artefact, which means pinned dependencies and a controlled build environment.
- **Fast enough that nobody avoids it.** Ten minutes is a common target for the path to a merge decision. Beyond that people batch changes to avoid waiting, which undoes continuous integration.
- **The only path to production**, so that what is running can always be traced to a commit.

Supply chain steps have become standard and are worth naming: scanning dependencies for known vulnerabilities, scanning built images, signing artefacts so their origin can be verified, and producing a software bill of materials listing what went into the build. **Provenance** is the one people miss: a signature says the artefact is authentic, while a provenance attestation records *how and where* it was built, which is what lets a consumer check it came from the expected pipeline rather than someone's laptop.

13. Release strategies

The system design note lists these; this is how they behave in practice.

Strategy	Mechanism	Rollback	Cost
Rolling	Replace instances in batches	Roll forward or redeploy the old version	None extra, but two versions run at once
Blue-green	Two full environments, switch traffic	Switch traffic back, provided the old version is still compatible with the data and state	Double the resources during the switch
Canary	Small share of traffic first, then widen	Route back, affecting few users	Needs traffic splitting and good metrics

Strategy	Mechanism	Rollback	Cost
Feature flag	Deploy off, enable separately	Turn the flag off	Flag lifecycle management

Rolling is the default and carries one requirement people forget: because two versions run simultaneously, every change has to be compatible with the version beside it, in both directions.

Canary needs enough traffic to be meaningful. Sending one percent of a hundred requests a minute to a new version produces no signal, so the comparison has to be against a metric with enough volume to move.

Deployment and release are different things, and separating them is worth stating explicitly because it is what feature flags actually buy. Deploying puts code on the servers; releasing makes the behaviour reachable by users. With a flag, code can be deployed on Tuesday and released on Thursday, and turned off again without another deployment.

Rollback and roll-forward are the two recovery options, and rollback is not automatically the safer one. Rolling back returns to a previous version, which is fast and only works if that version is still compatible with the current data. Rolling forward ships a fix, which is slower and is sometimes the only correct choice once a migration has changed data in a way the old version cannot read.

Database migrations constrain all of this, and this is the part that goes wrong. A rollback cannot undo a migration that dropped a column, so schema changes follow **expand, migrate, contract**: add the new shape, move the data and the reads, and remove the old shape in a later release once nothing needs it. Every step in that sequence has to work with both application versions, which is what makes it suit rolling deployments in particular.

14. Infrastructure as code

Declare the desired state of infrastructure in files, keep them in version control, and let a tool make reality match. The benefits are the same as for application code: review, history, repeatability, and the ability to rebuild an environment from nothing.

Two families are worth separating. **Provisioning** tools such as Terraform create and destroy resources from a declarative description. **Configuration management** tools such as Ansible converge the state of machines that already exist. Cloud-native environments often favour **immutable infrastructure**, where a change means replacing the instance rather than modifying it, which removes the drift that configuration management exists to correct.

```
resource "aws_db_instance" "orders" {
  identifier      = "orders-${var.environment}"
  engine          = "postgres"
  instance_class  = var.db_instance_class
  allocated_storage = 100
  multi_az        = var.environment == "production"

  tags = {
    Service = "orders"
    Owner   = "payments-team"      # tagging is what makes cost attributable
  }
}
```

State is the part that causes trouble. Terraform keeps a state file recording its mapping between the resources declared in code and the real infrastructure it manages. The configuration is the desired state; the state file is the bookkeeping that connects the two. Three things follow:

- **Concurrent changes have to be coordinated.** A shared remote backend with locking is the usual team arrangement, and the underlying requirement is that two applies cannot run against the same state at once.
- It frequently **contains secrets** in plain text, so it needs the same protection as any credential store.
- **Losing it means reconciling by hand**, because Terraform no longer knows which real resources correspond to which declarations, even though the resources themselves are unaffected.

Drift is reality diverging from the code, usually because someone changed something in the console during an incident. A `plan` reports it, and the resolution is either to apply the code over it or to bring the change into the code. Drift that is never reconciled is how infrastructure as code quietly becomes documentation.

15. Configuration and secrets

Configuration that varies between environments belongs **outside the artefact**, which is the config principle from the twelve-factor app: the same image runs in staging and production, and the environment supplies the difference. Baking environment-specific values into the build means one artefact per environment, which breaks build-once-promote-many.

Secrets need more than that. The rules are short and absolute enough to state plainly:

- Never in version control, including in history. A secret committed and then removed is still in the history and must be rotated rather than deleted.
- Never in an image layer, for the same reason.
- Never in a log line, which is how they most often escape.

The mechanism is a **secret manager** such as Vault, AWS Secrets Manager or a cloud key management service, with values injected at run time and rotated on a schedule.

Kubernetes Secrets are not encrypted by default

A `Secret` stores its values **base64 encoded**, which is an encoding rather than encryption and is trivially reversible. Encryption at rest for the cluster datastore has to be configured separately, and without it anyone who can read that datastore or has broad API access can read every secret in it.

The practical position: treat the object as a delivery mechanism, protect it with encryption at rest and tight access control, and add an external secret manager when centralised rotation, auditing and lifecycle management across more than one cluster are what is needed.

In Spring Boot the application side is straightforward: externalised configuration with profiles per environment, and `spring.config.import` to pull values from a secret manager at startup rather than shipping them.

16. Service levels and signals

Three terms that get used interchangeably and are not the same thing:

Term	What it is	Example
SLI , Service Level Indicator	A measurement of actual behaviour	The proportion of requests that succeed
SLO , Service Level Objective	An internal target for that measurement	99.9% over a rolling 30 days
SLA , Service Level Agreement	An external commitment, usually with consequences	99.9% availability, with service credits if missed

The relationship matters: the SLA is set looser than the SLO on purpose, so the team reacts when the objective is at risk rather than when the contract is. An availability figure with no indicator and no window attached is not yet a target.

Availability and latency are separate objectives rather than one composite. A service can be returning successful responses while missing its latency target, and the response to each is different.

The three signals divide the work of answering different questions, and the distributed systems note covers them in more depth:

Signal	Answers
Metrics	What is happening now, in aggregate, and is it within the objective
Logs	What happened in this specific request or process
Traces	Where the time went across services

What makes them useful together is a **correlation identifier** carried through every service and attached to every log line, so a trace showing a slow hop can be joined to the logs from exactly that hop. Without it, three signals are three separate haystacks.

17. Alerting and on-call

The single most useful principle: **alert on symptoms, not causes**.

Processor use at 90% may be perfectly healthy, and paging someone for it trains them to ignore pages. Error rate rising, latency breaching its objective, or a queue growing without bound are things users feel, and they are worth waking someone for whatever the underlying cause turns out to be.

Four questions every alert should survive:

- **What does it mean?** If the responder has to work that out from the alert name, it will cost minutes at the worst moment.
- **Who owns it?** An alert routed to everyone is owned by nobody.
- **What should they do?** A link to a runbook, even a short one.
- **Why now?** If it can wait until the morning, it is a ticket rather than a page.

Alert fatigue is the failure mode that matters. Too many alerts means people stop reading them, and then the one that mattered is missed among the noise. A page that fires and needs no action is a defect in the alert, and deleting alerts is legitimate maintenance.

The modern approach from SRE is **error budget burn rate** alerting: rather than paging on every breach, page when the objective's budget is being consumed fast enough to exhaust it. That distinguishes a brief blip from a sustained problem without needing a human to make the judgment.

On-call needs a rotation people can sustain: predictable schedules, a real handover, a clear escalation path, and compensation. If a rotation is exhausting, the answer is fixing the system that generates the pages rather than rotating people through it faster.

18. Incident response

Roles matter more than process during an incident, and the key one is that **the incident commander coordinates and does not fix**. Someone deep in a terminal cannot also track what has been tried, who is doing what, and when the next update is due.

Role	Responsibility
Incident commander	Coordinates, decides, holds the timeline. Does not debug
Communications lead	Updates stakeholders and status pages on a cadence
Subject matter experts	Investigate and fix, one area each

Two habits that separate a smooth incident from a chaotic one.

Mitigate first, diagnose second. Restoring service and understanding the cause are different goals, and the first one is more urgent. Rolling back, shifting traffic, or disabling a feature is often the right first move even when the cause is unknown, and diagnosis is easier afterwards without the pressure.

Communicate on a cadence, even with nothing new. Silence is read as nothing being done, and it generates interruptions that slow the people working.

The postmortem is blameless, and that is a practical choice rather than a kind one. A person who expects consequences will describe what happened carefully and incompletely, and the incomplete part is usually the most valuable. The goal is finding the systemic conditions that let a normal human action cause an outage.

Action items need an owner and a date, or the document becomes a record of intentions. "Five whys" is a common technique for reaching past the immediate trigger, with the caveat that real incidents usually have several contributing causes rather than one chain.

19. Error budgets and toil

An error budget turns reliability from an argument into arithmetic. If the objective is 99.9% of requests succeeding over 30 days, the budget is the remaining 0.1%. Spending it is not a failure, it is the point: the budget exists to be used on shipping change.

What makes it clever is the incentive it creates. While the budget is intact, the team ships freely. When it is exhausted, the team's agreed policy takes over, which commonly means restricting feature work and prioritising reliability work until the budget recovers. The freeze is the policy rather than the definition, and organisations set different ones. Either way both sides of the old development-against-operations argument now share one number, and neither can win by pushing the other around.

100% is the wrong target. The cost of each additional nine rises sharply, and past a point users cannot tell, because their own network and device are less reliable than the service. Choosing the objective is a product decision about what users notice and what the business will pay for.

Toil is manual, repetitive work that could be automated, scales with the size of the service, and leaves nothing behind when it is done. Restarting a service by hand every week is toil; writing the thing that stops it needing a restart is engineering.

The SRE practice is to measure it and cap it, with Google's convention being no more than half a person's time, so that the remainder goes into work that reduces future toil. Without a cap, a growing service absorbs the whole team into keeping it alive, and the capacity to improve it disappears.

20. Interview questions

Each answer below is written to be said out loud as it stands. The last sentence always leads toward the next question rather than closing the topic.

"What is the difference between continuous delivery and continuous deployment?"

Continuous delivery means every change that passes the pipeline is releasable, so the artefact is built, tested and ready, and deploying is a business decision someone makes by pressing a button. Continuous deployment removes that button, so anything passing the pipeline goes to production automatically. Both sit on top of continuous integration, which is frequent integration into a shared mainline with every integration automatically verified, and the value there is the frequency rather than the automation, because integrating small changes continuously removes the painful merge at the end of a long branch. What makes continuous deployment workable is the safety net around it: good tests, a canary or gradual rollout, and monitoring that can trigger a rollback, because without those the button was doing real work. Continuous delivery is the more common resting point in regulated environments, and that is a defensible choice rather than a failure to finish.

"How is a container different from a virtual machine?"

A container is a process on the host's kernel with two kinds of isolation around it: namespaces controlling what it can see, so its own process IDs, network and mounts, and cgroups controlling what it can use, so processor, memory and input-output. A virtual machine virtualises hardware and runs its own kernel. The consequences are that containers start in well under a second and carry almost no overhead, while the isolation is weaker because the kernel is shared, so a kernel vulnerability crosses the boundary. The Java-specific detail I would add is that older JVMs read the host's memory rather than the cgroup limit, so a container capped at 512 megabytes would size its heap for the whole machine and get killed. That has been fixed since JDK 10, but the lesson still applies, which is to set the heap as a percentage with `MaxRAMPercentage` rather than matching `Xmx` to the container limit, since the JVM needs metaspace, thread stacks and direct buffers outside the heap.

"What do requests and limits do in Kubernetes, and what happens when they are wrong?"

The request is what the scheduler uses when deciding which node a pod goes on and how much capacity is left, and the limit is the ceiling once it is running. The important part is that the two resources behave differently when the limit is hit: exceeding the processor limit throttles the container so it runs slowly, while exceeding the memory limit can cause the kernel to kill the process, because memory cannot be throttled the way processor time can. That asymmetry means memory limits deserve more care, and it is why a Java service that sizes its heap to the container limit gets killed with exit code 137 and no stack trace. If requests are set too low the scheduler overcommits the node and pods get evicted under pressure, and if they are set too high the capacity is reserved and wasted. The related mistake I would mention is a liveness probe that checks a downstream dependency, because when that dependency blips every replica fails liveness at once and the platform restarts all of them.

"Would you use Kubernetes for this system?"

It depends on how many services and teams there are, and I would push back on it as a default. Kubernetes carries real operational cost: cluster lifecycle and upgrade management, networking and ingress, access control, resource tuning, and a surface area everyone touching it has to learn. It earns that when there are many services and several teams needing to deploy independently, when bin-packing workloads onto shared nodes saves real money, or when there is a platform team whose job is running it. For a handful of services owned by one team with steady load, a managed container platform like Cloud Run or ECS gives most of the deployment and scaling benefit for a fraction of the surface. It is a good answer to a problem a lot of organisations adopting it do not have yet.

"How do you handle secrets?"

Through a secret manager, with values injected at run time and rotated on a schedule, and never in three places: version control, image layers, or log lines. The version control one has a detail worth stating, which is that a secret committed and then deleted is still in the history, so the response is rotating it rather than removing the file. The thing I would flag specifically is that Kubernetes Secrets are base64 encoded rather than encrypted, which is an encoding and trivially reversible, so without encryption at rest enabled on the cluster datastore anyone who can read that datastore can read every secret. I treat the Secret object as a delivery mechanism rather than protection, with a real secret manager behind it. Configuration that is not secret still belongs outside the artefact, so the same image runs everywhere and the environment supplies the difference.

"What is an error budget?"

It turns reliability from an argument into arithmetic. If the objective is 99.9 percent of requests succeeding over 30 days, the budget is the other 0.1 percent, and spending it is the point rather than a failure. What makes it useful is the incentive: while the budget is intact the team ships freely, and when it is exhausted the agreed policy takes over, which commonly means restricting feature work and prioritising reliability until it recovers. I would be careful to call that a policy rather than the definition, since organisations set different ones, but either way development and operations end up working from one number instead of arguing about how careful to be. It also makes clear why 100 percent is the wrong target, since each additional nine costs sharply more and past a point users cannot tell, because their own network and device are less reliable than the service. The related practice is capping toil, which is manual repetitive work that scales with the service and leaves nothing behind, because without a cap a growing service absorbs the whole team into keeping it alive.

"Walk me through what you do when production breaks."

Mitigate first and diagnose second, because restoring service and understanding the cause are different goals and only one of them is urgent. So rolling back, shifting traffic away, or turning off a feature is often the right first move even before the cause is known, and diagnosis is easier afterwards without the pressure. On the process side the thing that matters most is that the incident commander coordinates rather than fixes, because someone deep in a terminal cannot also track what has been tried and who is doing what, and there should be someone communicating on a cadence even when there is nothing new, since silence gets read as nothing happening and generates interruptions. Afterwards the postmortem is blameless for a practical reason rather than a kind one: a person expecting consequences describes what happened incompletely, and the missing part is usually the most valuable. Action items need an owner and a date or the document is just a record of intentions.

"How would you measure a team's delivery performance?"

The four DORA metrics, which are deployment frequency, lead time from commit to production, change failure rate, and time to restore service. The first two measure speed and the last two stability, and the interesting finding is that they are not opposed, so teams that do well on one tend to do well on the other, which contradicts the intuition that shipping faster necessarily breaks more. The causal story is that small frequent changes are easier to review, verify and reverse. I would add two qualifications rather than wait to be challenged on them: they measure delivery performance and not whether the software is worth building, and they are gameable, since deployment frequency rises the moment one deploy is split into ten. So they work well as a trend a team watches for itself and badly as a target imposed on it.

21. Common mistakes

- Treating DevOps as a job title or a separate team, which rebuilds the handoff between development and operations that the practice exists to remove.
- Using DORA metrics as targets imposed on a team rather than a trend the team watches, which makes them easy to game and stops them measuring anything.
- Assuming the cloud provider is responsible for security misconfiguration. The provider secures the cloud; the customer secures what is in it.
- Ignoring data transfer costs, particularly outbound and cross-zone, then discovering that an availability decision generated most of the bill.
- Setting `-Xmx` equal to a container memory limit, leaving nothing for metaspace, thread stacks and direct buffers, so the kernel kills the process with no Java error to explain it.
- Copying source before resolving dependencies in a Dockerfile, so every source change re-downloads every dependency.
- Baking a secret into an image layer, where it stays in the history even after a later layer deletes it.
- Deploying the `latest` tag, so the thing tested and the thing running may differ with no record of which is which.

- Putting a dependency check in a liveness probe, so a brief outage in that dependency restarts every replica at once.
- Setting Kubernetes memory limits without understanding that memory is killed rather than throttled, unlike processor time.
- Rebuilding the artefact per environment, so the thing tested is not the thing shipped.
- Running a slow pipeline, so people batch changes to avoid waiting, which removes the benefit of continuous integration.
- Treating a Kubernetes Secret as encrypted. It is base64 encoded, which is reversible by anyone who can read it.
- Keeping Terraform state locally or unlocked, so two concurrent applies corrupt it, or losing it and having to reconcile by hand.
- Fixing something in the console during an incident and never bringing the change back into code, so the code quietly becomes documentation.
- Alerting on causes such as processor use rather than symptoms users feel, which trains people to ignore pages.
- Dropping a column in a migration and assuming the release can be rolled back, when the data is gone.

22. Quick reference

Term	Meaning
IaaS, PaaS, FaaS, SaaS	Progressively more managed by the provider, less controlled by us
Shared responsibility	The provider secures the cloud; the customer secures what is in it
Availability zone	An isolated data centre within a region, with independent power and network
Egress	Outbound data transfer, which is charged where inbound usually is not
Spot capacity	Much cheaper, reclaimable with little notice. For interruptible work
Namespaces and cgroups	What a container can see, and what it can use
Exit code 137	Terminated by SIGKILL, most often an out-of-memory kill against a container limit
<code>MaxRAMPercentage</code>	Sizes the JVM heap as a share of the container limit, not the host
Multi-stage build	Compile in one stage, ship only the artefact in the final image
Request against limit	What the scheduler uses for placement, against the ceiling when running
Processor against memory limit	Over the first throttles; over the second kills
Liveness, readiness, startup	Restart it, stop routing to it, and hold off while it starts
Continuous integration	Frequent integration into the mainline, with automated verification

Term	Meaning
Public against private subnet	A routing property: whether the route table reaches an internet gateway
Security group against network ACL	Stateful, attached to a resource, against stateless, at the subnet
SLI, SLO, SLA	The measurement, the internal target, the external commitment
Continuous delivery	Every passing change is releasable. A human presses the button
Continuous deployment	Every passing change is released. No button
Build once, promote	The tested artefact is the shipped artefact
Expand, migrate, contract	A backward-compatible schema migration sequence, suited to rolling deployments
Deployment against release	Code on the servers, against behaviour reachable by users
Rollback against roll-forward	Return to the old version, against shipping a fix. Rollback is not always the safe one
Terraform state	Terraform's mapping from declarations to real infrastructure. Coordinate access, treat as sensitive
Drift	Reality diverging from the code, usually via a console change
Immutable infrastructure	Replace instances rather than modifying them
DORA	Deployment frequency, lead time, change failure rate, time to restore
Error budget	The allowed unreliability. Exhausting it triggers whatever policy the team agreed
Toil	Manual repetitive work that scales with the service and leaves nothing behind
Blameless postmortem	Blameless so people describe what happened completely

Symptom	Likely cause
Pod terminating with exit code 137	Container memory limit exceeded, and the kernel killed it
A Java service dies with no stack trace	The same, usually <code>-Xmx</code> set to the container limit
Every replica restarting when a dependency blips	A dependency check inside a liveness probe
Docker builds slow after every source change	Source copied before dependencies were resolved
It worked in staging and not production	The artefact was rebuilt rather than promoted, or config differs
Cloud bill dominated by an unexplained line	Data transfer, usually egress or cross-zone
Terraform apply destroying something unexpectedly	Drift from a console change, or a stale or shared state file
A rollback that does not restore service	A migration already dropped or transformed the data

Symptom	Likely cause
Pages nobody acts on	Alerting on causes rather than symptoms
A team with no time for improvement work	Toil is uncapped and growing with the service